task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py | task5_output.txt | task6.py

```python
import threading
import time
THREADS = 4
INCREMENTS = 2_000_000
EXPECTED = THREADS * INCREMENTS
counter = 0
locked_counter = 0
counter_lock = threading.Lock()
def increment_without_lock():
    global counter
    for _ in range(INCREMENTS):
        counter += 1

def increment_with_lock():
    global locked_counter
    for _ in range(INCREMENTS):
        with counter_lock:
            locked_counter += 1

def run_test(target):
    threads = []
    start_time = time.perf_counter()
    for _ in range(THREADS):
        thread = threading.Thread(target=target)
        threads.append(thread)
        thread.start()
    for thread in threads:
        thread.join()
    return time.perf_counter() - start_time

```

task1_output.txt | task2.py | task2_output.txt | task3.py | task3_output.txt | task4.py | task4_output.txt | task5.py | task5_output.txt | task6.py

```python


def main():
    global counter, locked_counter
    print("TASK 6: SYNCHRONIZATION COST & DATA LOSS")
    print(f"Number of threads: {THREADS}")
    print(f"Increments per thread: {INCREMENTS}")
    print(f"Expected final value: {EXPECTED}")
    print()
    counter = 0
    time_without_lock = run_test(increment_without_lock)
    lost_updates = EXPECTED - counter
    lost_percentage = (lost_updates / EXPECTED) * 100

    print("VERSION 1: WITHOUT LOCK")
    print(f"Expected final value: {EXPECTED}")
    print(f"Actual final value: {counter}")
    print(f"Lost updates: {lost_updates}")
    print(f"Percentage of updates lost: {lost_percentage:.6f}%")
    print(f"Execution time: {time_without_lock:.6f} seconds")
    print()
    locked_counter = 0
    time_with_lock = run_test(increment_with_lock)

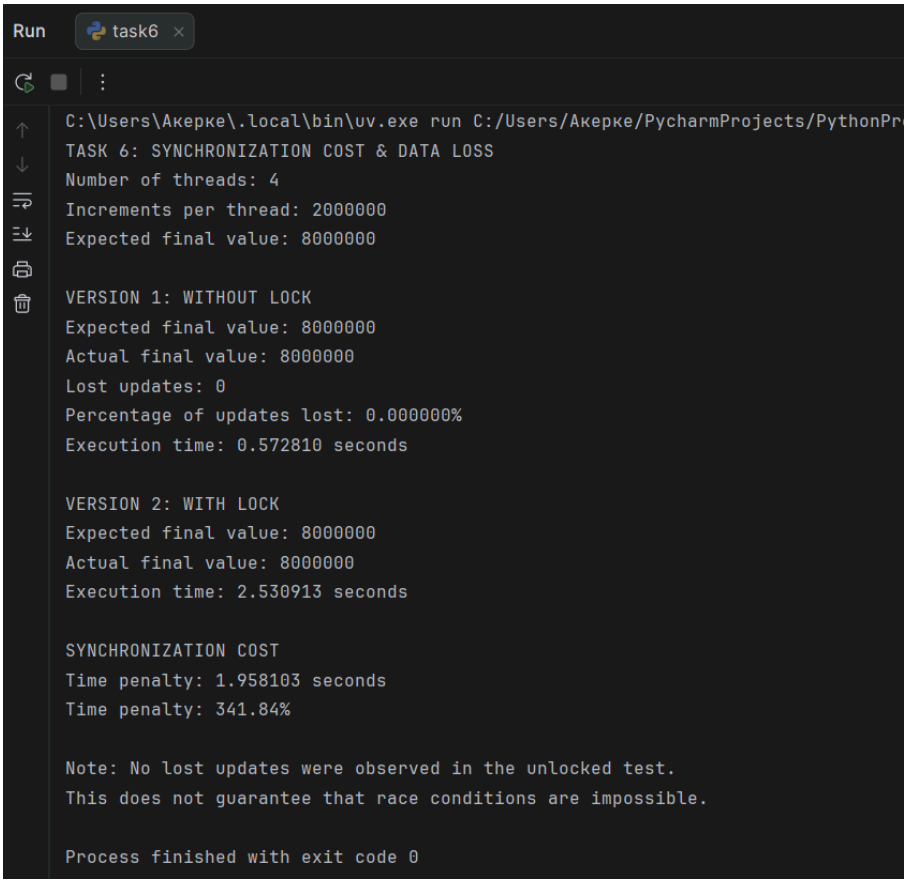
    print("VERSION 2: WITH LOCK")
    print(f"Expected final value: {EXPECTED}")
    print(f"Actual final value: {locked_counter}")
    print(f"Execution time: {time_with_lock:.6f} seconds")

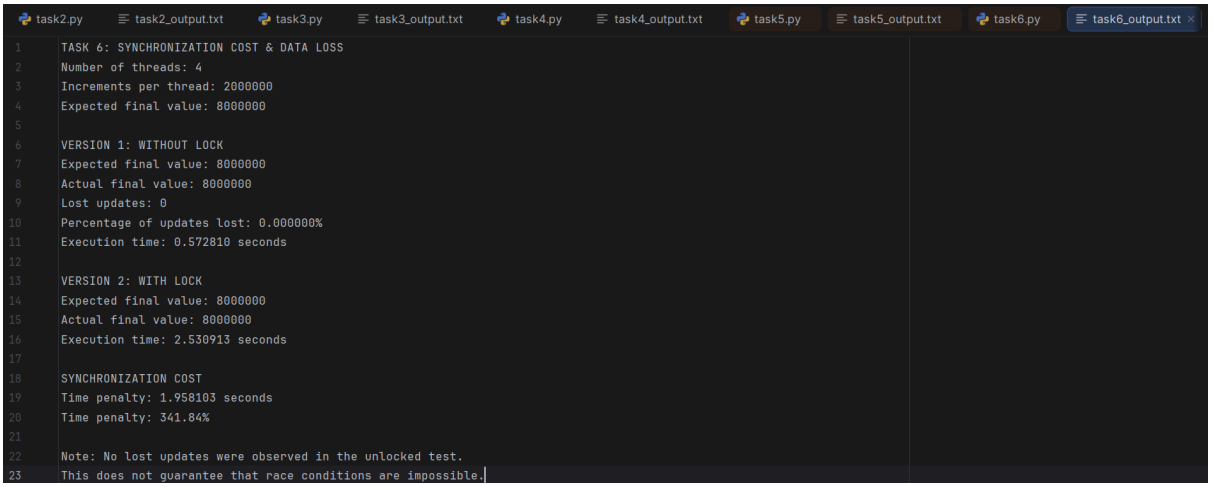
    penalty = time_with_lock - time_without_lock
    penalty_percentage = (
        (penalty / time_without_lock) * 100
        if time_without_lock > 0
        else 0
    )

    print()
    print("SYNCHRONIZATION COST")
    print(f"Time penalty: {penalty:.6f} seconds")
    print(f"Time penalty: {penalty_percentage:.2f}%")

    if counter == EXPECTED:
        print()
        print("Note: No lost updates were observed in the unlocked test.")
        print("This does not guarantee that race conditions are impossible.")


if __name__ == "__main__":
    main()
```


Run
task6
C:\Users\Акерке\.local\bin\uv.exe run C:/Users/Акерке/PycharmProjects/PythonPr
TASK 6: SYNCHRONIZATION COST & DATA LOSS
Number of threads: 4
Increments per thread: 2000000
Expected final value: 8000000
VERSION 1: WITHOUT LOCK
Expected final value: 8000000
Actual final value: 8000000
Lost updates: 0
Percentage of updates lost: 0.000000%
Execution time: 0.572810 seconds
VERSION 2: WITH LOCK
Expected final value: 8000000
Actual final value: 8000000
Execution time: 2.530913 seconds
SYNCHRONIZATION COST
Time penalty: 1.958103 seconds
Time penalty: 341.84%
Note: No lost updates were observed in the unlocked test.
This does not guarantee that race conditions are impossible.
Process finished with exit code 0


task2.py
task2_output.txt
task3.py
task3_output.txt
task4.py
task4_output.txt
task5.py
task5_output.txt
task6.py
task6_output.txt
TASK 6: SYNCHRONIZATION COST & DATA LOSS
Number of threads: 4
Increments per thread: 2000000
Expected final value: 8000000
VERSION 1: WITHOUT LOCK
Expected final value: 8000000
Actual final value: 8000000
Lost updates: 0
Percentage of updates lost: 0.000000%
Execution time: 0.572810 seconds
VERSION 2: WITH LOCK
Expected final value: 8000000
Actual final value: 8000000
Execution time: 2.530913 seconds
SYNCHRONIZATION COST
Time penalty: 1.958103 seconds
Time penalty: 341.84%
Note: No lost updates were observed in the unlocked test.
This does not guarantee that race conditions are impossible.